

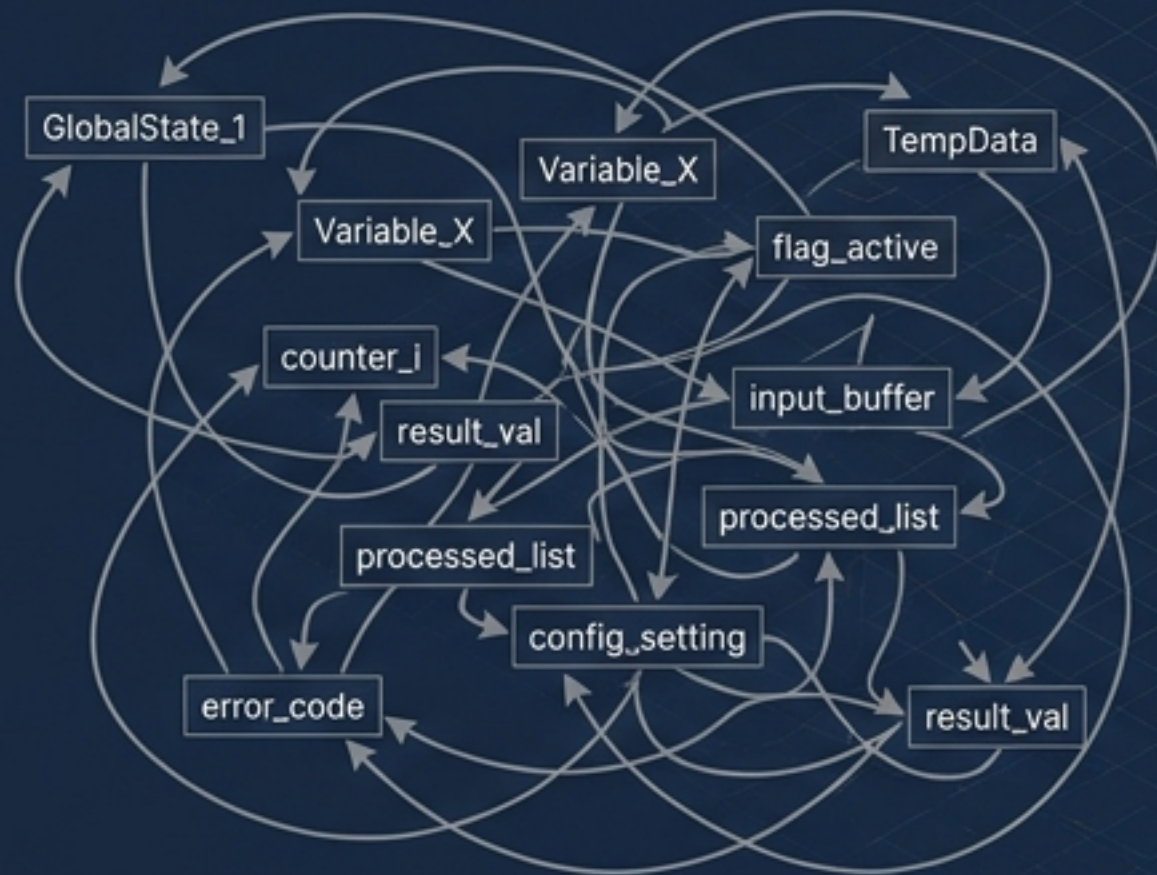


OBJECT-ORIENTED PROGRAMMING IN PYTHON 3

Architecting Systems from First Principles

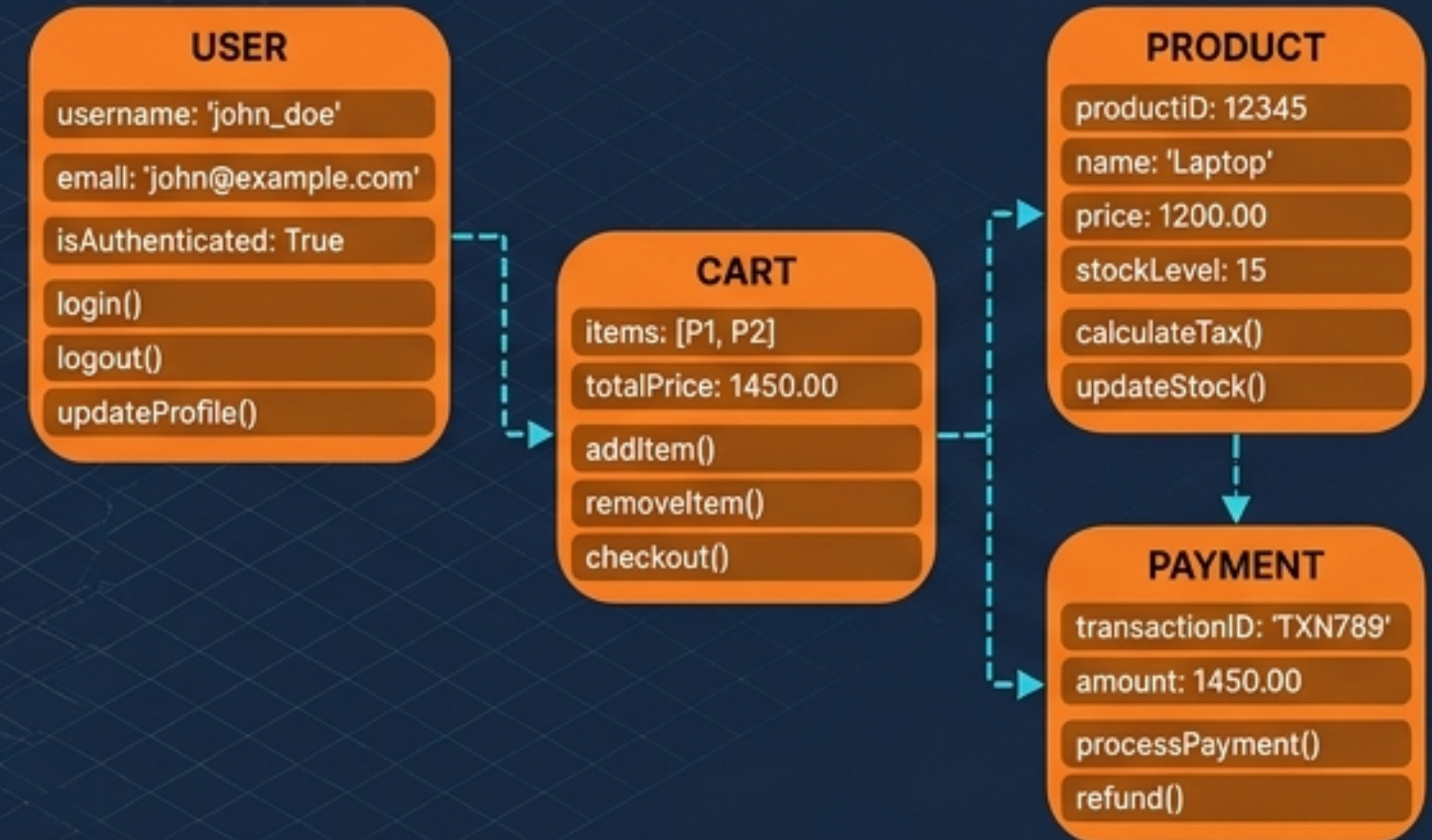
A Shift in Mental Models

Procedural Code



Focus: Functions and global data interaction.
Brittleness increases with scale.

Object-Oriented Design



Focus: Self-contained objects with internal state and behaviors.
Promotes modularity and reusability.

Procedural Limits

Linear sequences of instructions become tangled and brittle as programs scale.

A New Mindset

Reorganise code around entities and their states, not just execution steps.

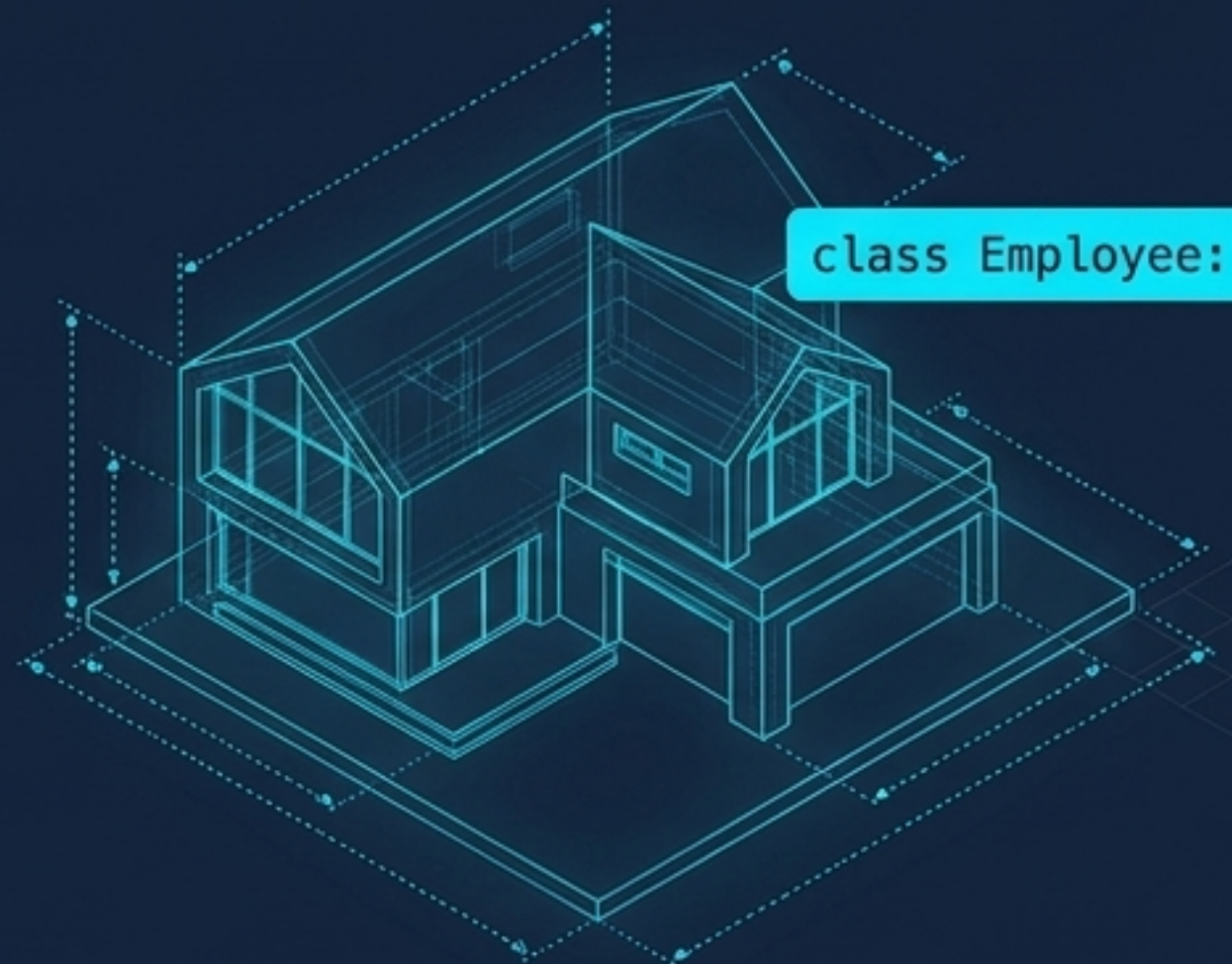
Modularity

Bundle data and the actions that operate on that data into self-contained units.

Everything is an Object

In Python, even integers, strings, and functions are objects. Example: '42'.bit_length() proves the integer 42 has built-in methods.

Design vs Reality: The Blueprint Analogy



The Class (Blueprint): A template dictating what an entity holds and does. It is a definition, not a real entity.



The Object (Instance): A concrete realisation of the class built in memory. Multiple distinct objects can be built from one single class.

Private State: Each object maintains its own independent copy of the data. Changes to employee1 do not affect employee2.

Translating Reality into Code

Real-World Entity	Nouns -> Attributes (State)	Verbs -> Methods (Behaviour)
 Employee	name, age, position, salary	increase_salary(), info(), get_department()
 BankAccount	owner, balance, account_number, currency	deposit(), withdraw(), get_balance(), transfer()

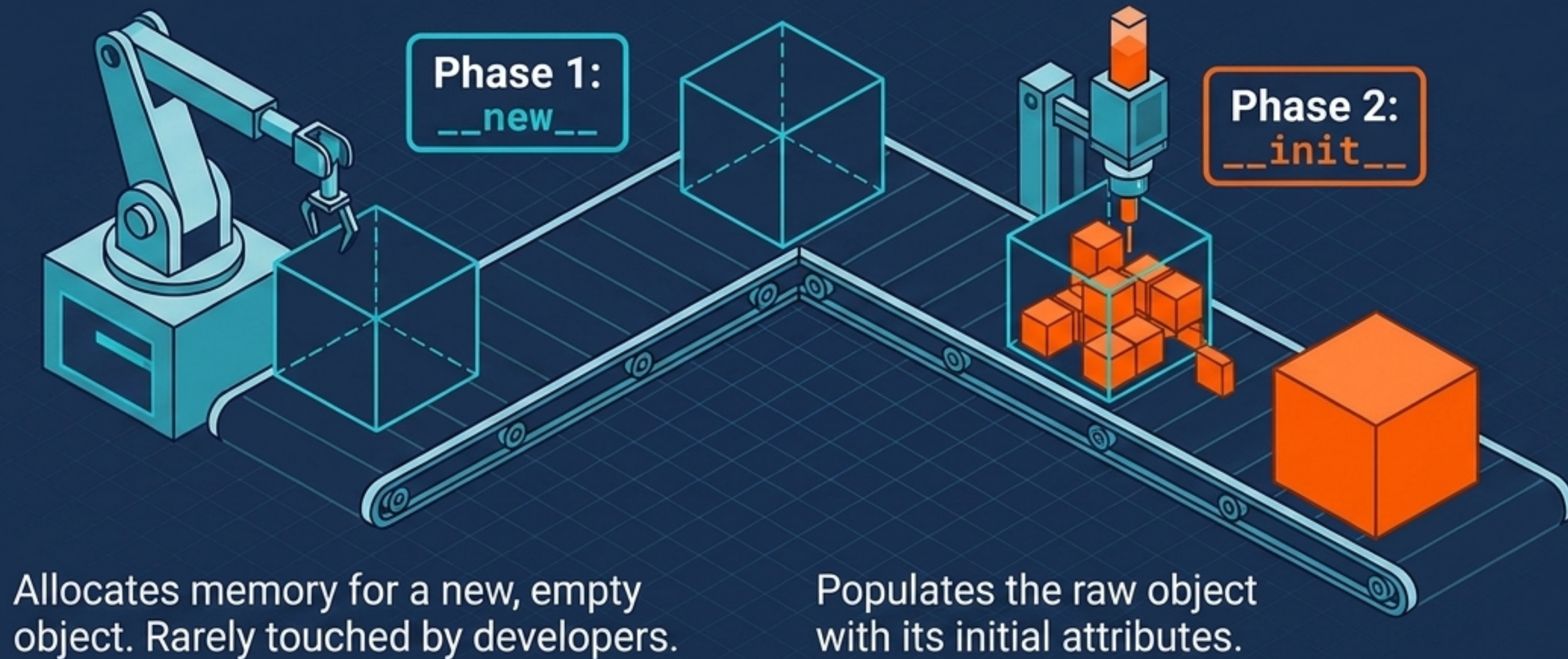
State Representation

An object's state is the sum of its current attribute values.

Dot Notation

Access state and behaviour intuitively using **object.attribute** or **object.method()**.

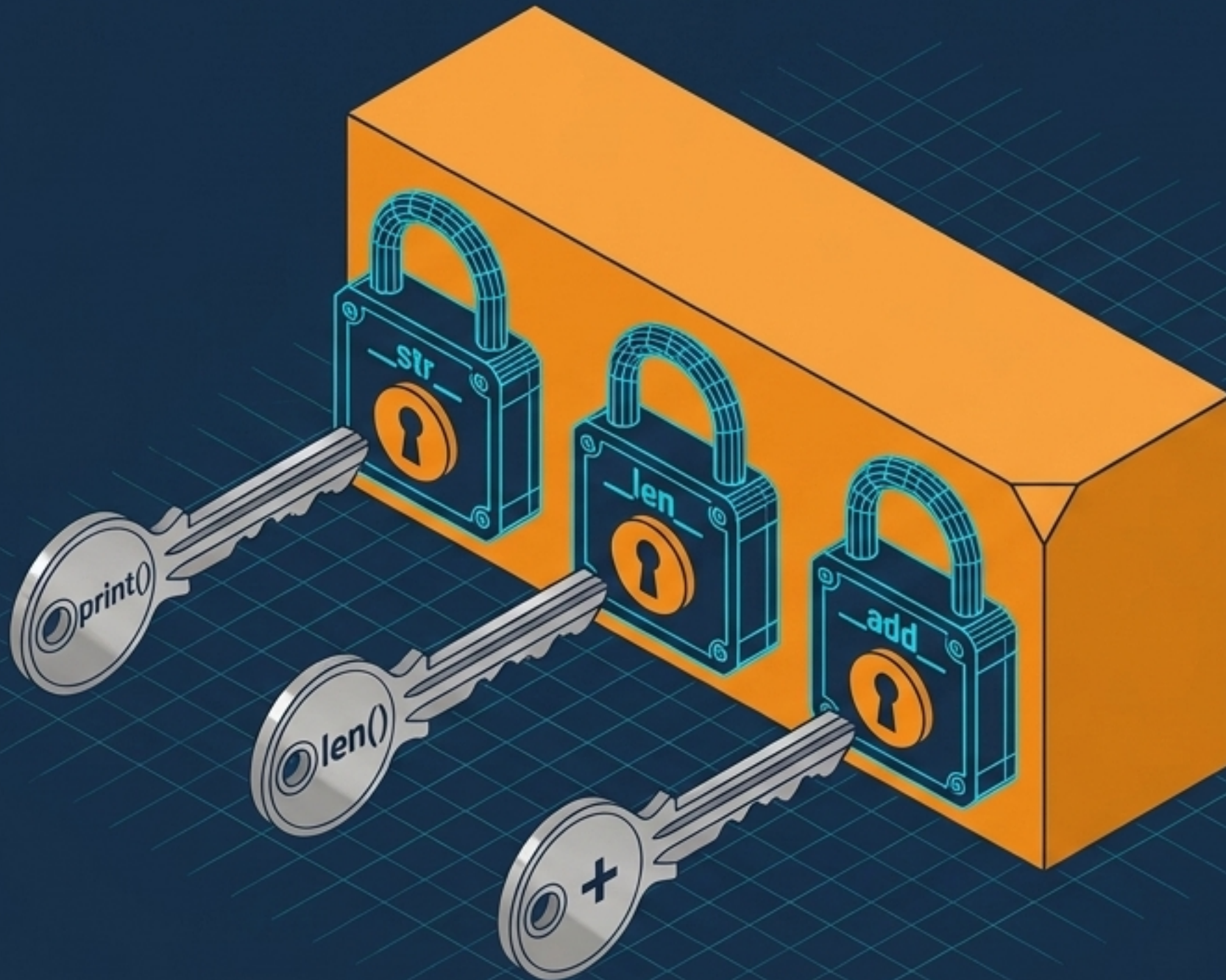
The Creation Pipeline: Allocating and Populating



The Guarantee: Ensures every object begins its life in a valid, predictable state.

The `self` Parameter: Python implicitly passes the newly created object as the first argument (`self`) so the method knows which object's data to populate.

Dunder Methods: Python's Magic Interface

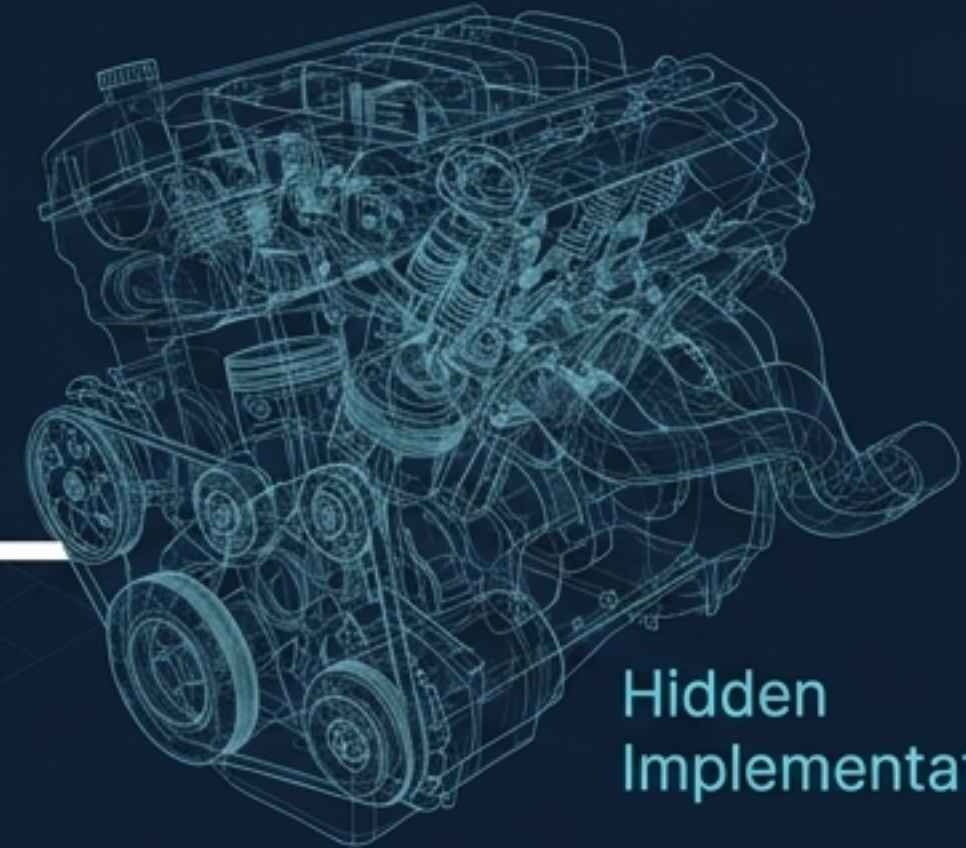


- **The Convention:** Double underscore methods are triggered automatically by built-in Python operations.
- **Human vs Developer:** `__str__` provides readable output for users; `__repr__` provides precise reconstruction for debugging.

```
__str__(self)  
  
Called by print() and str().  
Should return a human-readable  
string description  
of the object.  
  
Example: 'Employee: Ji-Soo  
(developer, age 36)'.
```
- **Operator Overloading:** Enable native behaviours like equality (`__eq__`), length (`__len__`), and custom addition (`__add__`).
- **Seamless Integration:** These methods allow your custom classes to blend in and behave exactly like native Python types.

Pillar 1: Abstraction

Simple Interface



Hidden
Implementation

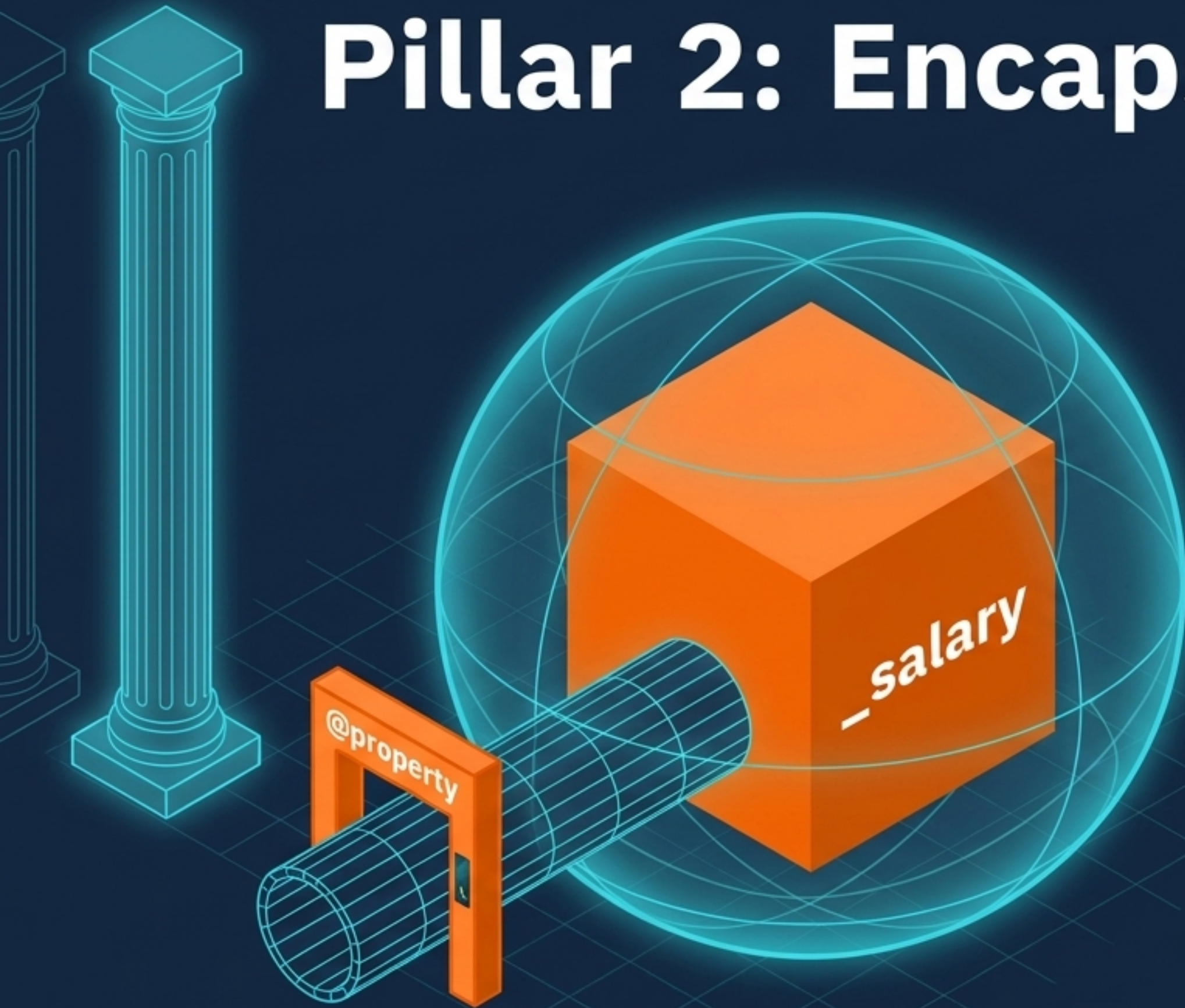
Core Principle: Hide complex implementation details behind a simplified, clean interface.

Example: Calling `employee.increase_salary(10)` completely hides the background math, database updates, and audit logging.

User Focus: The caller only needs to know what a method does, not how it achieves it.

The Benefit: You can completely rewrite the underlying engine logic without breaking the rest of your program.

Pillar 2: Encapsulation



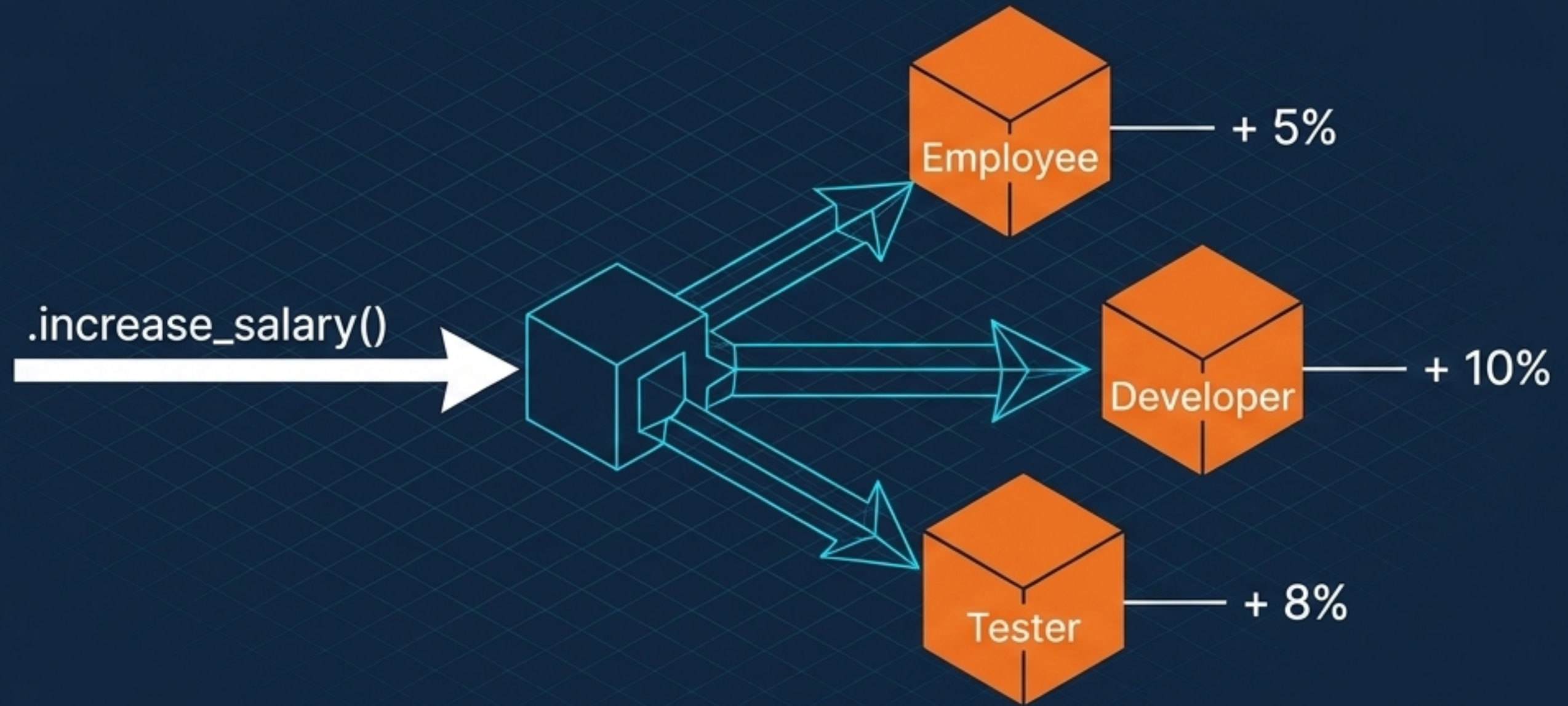
- **Core Principle:** Protect internal state from direct, unauthorised modification.
- **Pythonic Privacy:** A leading underscore (e.g., `_salary`) is the developer convention for 'internal use only'.
- **Controlled Access:** The `@property` decorator transforms methods into clean, attribute-like getters and setters.
- **The Benefit:** Enforce validation, business rules, and prevent catastrophic bugs (like `salary = -50000`) from a single entry point.

Pillar 3: Inheritance



- **Core Principle:** Create new classes by extending existing ones, strictly enforcing the DRY (Don't Repeat Yourself) principle.
- **The 'Is-A' Rule:** Inheritance only applies if a child IS-A parent. A Developer IS-A Employee.
- **Method Overriding:** Subclasses can redefine parent methods to introduce specialised, specific behaviours.
- **The `super()` Function:** Safely delegates initialisation back up to the parent class to handle inherited attributes.

Pillar 4: Polymorphism



Core Principle:

One interface, many behaviours. Different object types respond to the exact same method call in their own way.

Duck Typing:

Python focuses on the presence of a method rather than strict object lineage.

Dynamic Response:

A Developer and a Tester execute different mathematical caps to the same `.increase_salary()` call.

The Benefit:

Eliminates brittle, endlessly growing if/elif type-checking chains. Code scales elegantly.

Data Scope: Shared vs Private

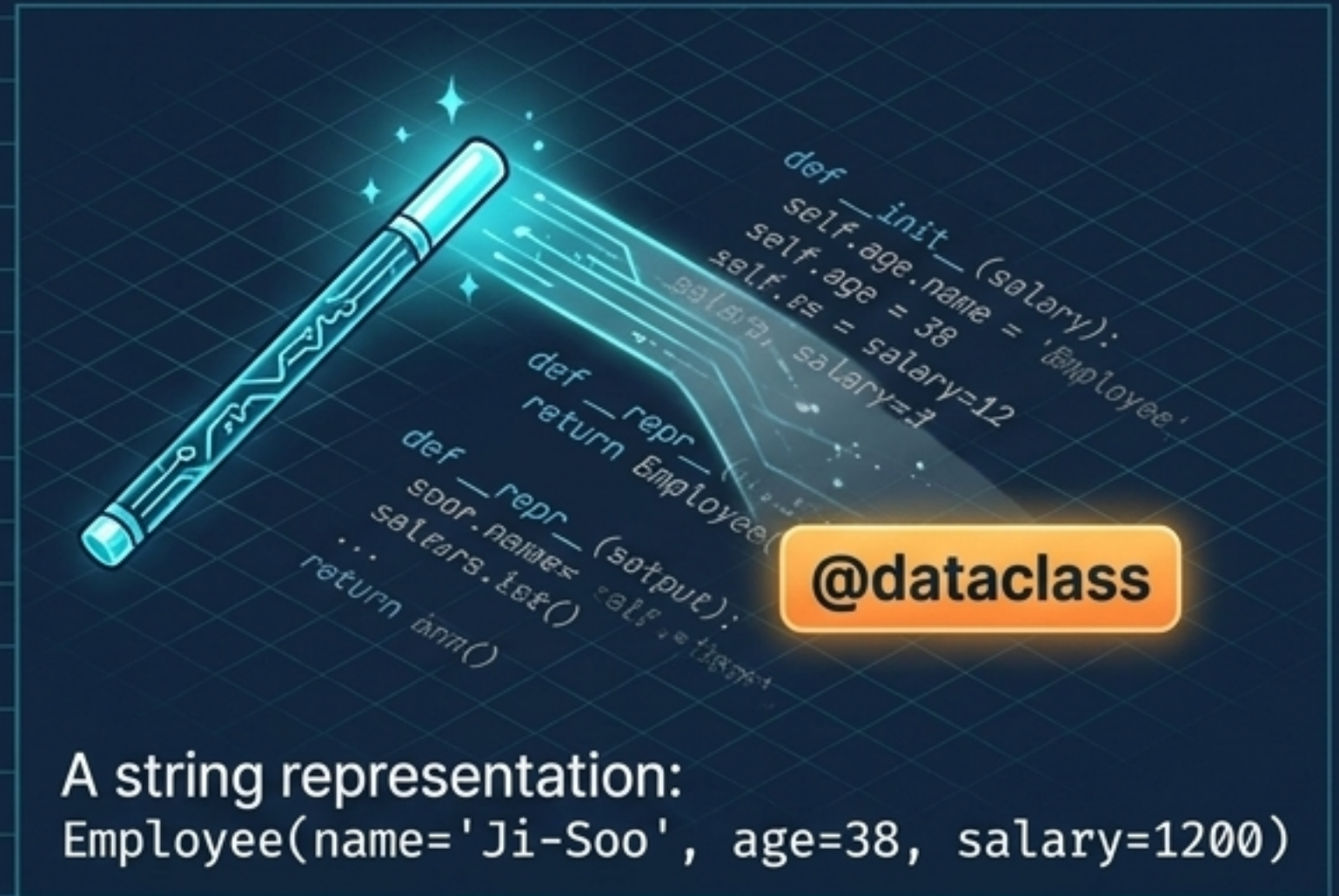
	Instance Attribute 	Class Attribute 
Definition	Inside <code>__init__</code> via <code>self</code> .	At class level, outside methods.
Storage	Unique private copy per object.	Single shared copy for all objects.
Use Case	Independent data (name, age).	Global templates or counters (company_name, employee_count).

Introspection Tool: Use `isinstance(obj, cls)` to inspect an object's blueprint dynamically at runtime.

Modern Python: Efficiency & Elegance

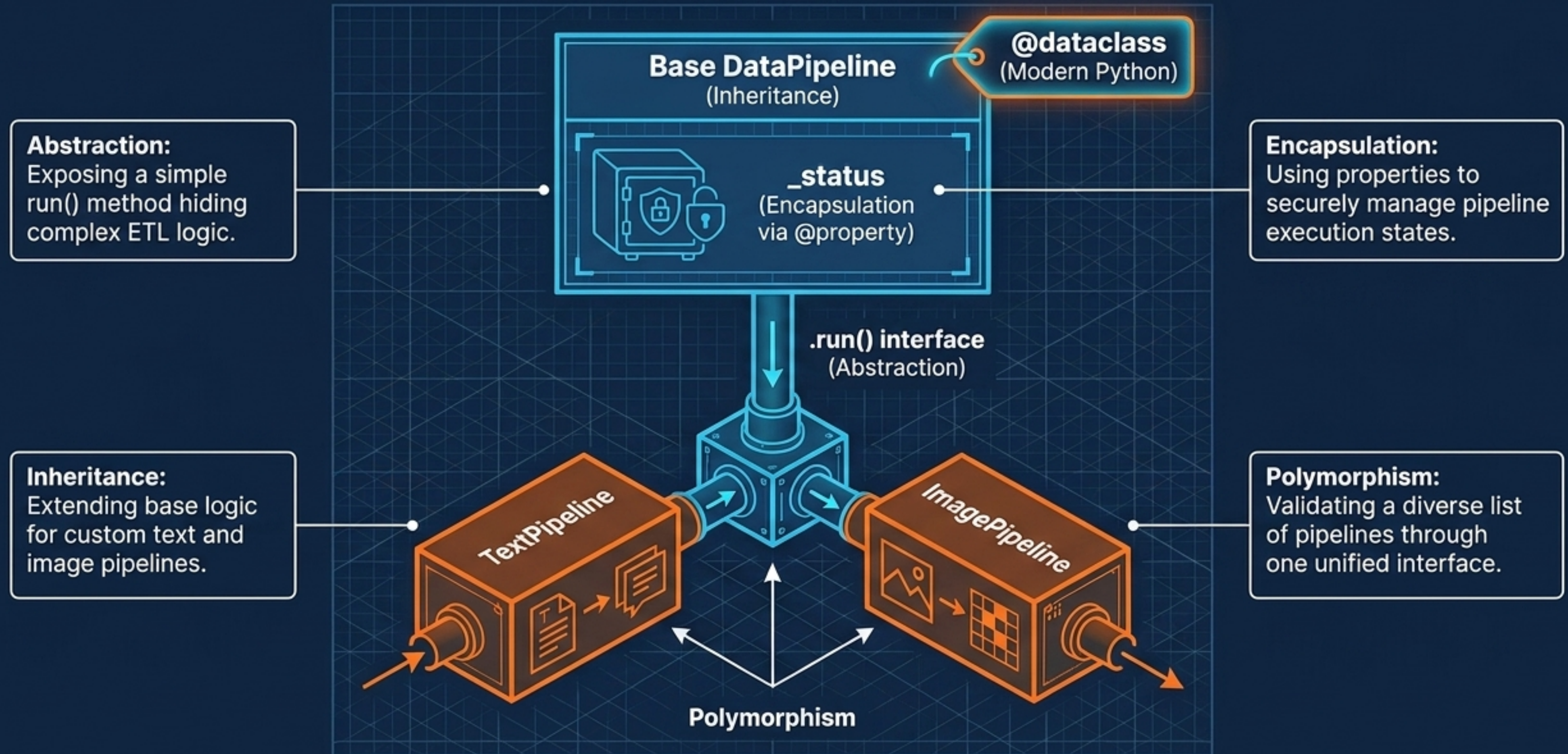


- **Memory Efficiency:** Using `__slots__` removes dictionary overhead, reducing an object's memory footprint by 40-60%.
- **Speed:** Direct slot access is noticeably faster for high-volume objects.

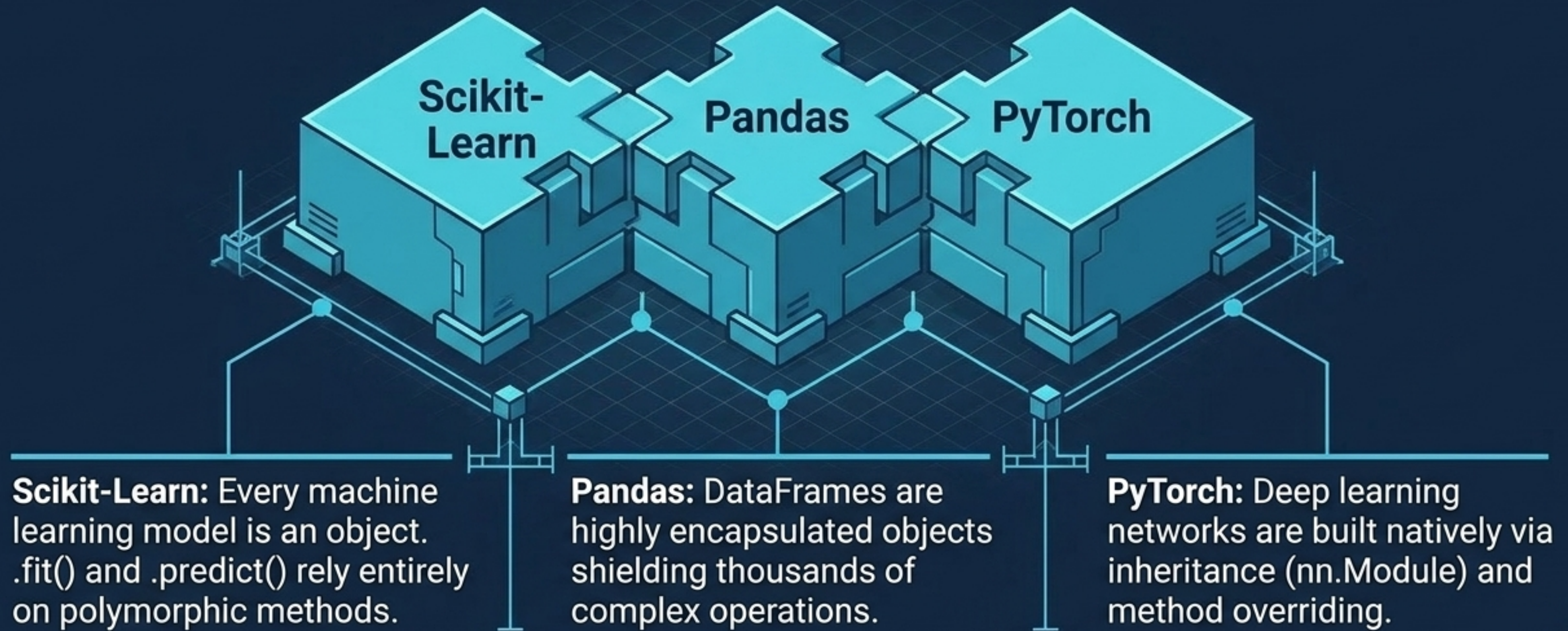


- **Data Classes (`@dataclass`):** Automatically generates `__init__`, `__repr__`, and `__eq__` based on type hints.
- **Focus on Data:** Reduces boilerplate and human error, perfect for data-heavy containers and records.

Synthesis: The Enterprise Data Pipeline



Why OOP is the Engine of Data Science



The Bottom Line: To build professional AI and data pipelines, you must speak the architectural language of the tools.

The Engineer's Mindset

Mental Shift: Think in **entities**, **modularity**, and **states**—not just sequences of commands.

The **self Link:** The fundamental glue connecting a specific object to its internal data and behaviours.



The Blueprint: **Classes** define structural potential; **Objects** bring it to life with independent memory.

The Pillars: **Abstraction** simplifies, **Encapsulation** protects, **Inheritance** reuses, and **Polymorphism** scales.

